

Lecture: Data Manipulation and Transformation

Actuarial Data Science - Open Learning Resource
Fei Huang, UNSW Sydney



Table of contents

- Introduction
- Data Manipulation Functions
- Relational Data



Recommended Reading

- *R for Data Science*, Chapters 3, 19
- Applied predictive Modelling, Chapters 3.3 (only transformations to resolve outliers), 3.4

Some coding examples in this lecture are adapted from *R for Data Science* (Wickham, Çetinkaya-Rundel, and Grolemund 2023).



Introduction



Motivation

- It is rare that you get the data in exactly the form you need.
- You may need to create new variables or summaries, or
- Simply rename variables or reorder observations to make the data easier to work with.

In practice, a large part of an actuary's work involves cleaning and reshaping data to ensure that it is trustworthy and easy to analyse.

In this lecture, we introduce a small set of powerful verbs (`filter()`, `select()`, `mutate()`, `summarise()`, `group_by()`, and joins) that can be combined to express many common data-wrangling tasks clearly.



Using R to Manipulate Data

- R package: `dplyr` (a core member of `tidyverse`) for data manipulation and transformation
 - Note: `dplyr` overwrites some functions in base R (e.g. `filter()`, `lag()`). To use the base versions (after loading `dplyr`), specify `stats::filter()` and `stats::lag()`.
- Data: the `nycflights13` package, which contains data on flights departing from New York City in 2013
- We use `ggplot2` to help us explore and understand the data.

```
1 #install.packages("nycflights13")
2 library(nycflights13)
3 library(tidyverse)
```



Data

```
1 flights #tibble, tweaked data frame to work better in tidyverse
```

year <int>	month <int>	day <int>	dep_time <int>	sched_dep_time <int>	dep_delay <dbl>
2013	1	1	517	515	2
2013	1	1	533	529	4
2013	1	1	542	540	2
2013	1	1	544	545	-1
2013	1	1	554	600	-6
2013	1	1	554	558	-4
2013	1	1	555	600	-5
2013	1	1	557	600	-3
2013	1	1	557	600	-3
2013	1	1	558	600	-2

1-10 of 10,000 rows | 1-6 of 19 columns

Previous 1 2 3 4 5 6 ... 1000 Next

```
1 view(flights) # will open the dataset in the RStudio viewer
```

See also: Filter Data



Types of variables

- `int`: integers
- `dbl`: doubles (real numbers)
- `chr`: character vectors (strings)
- `dtm`: date-times (a date + time)
- `lgl`: logical vectors (TRUE or FALSE)
- `fctr`: factors (categorical variables with fixed levels)
- `date`: dates.

These are the common variable types used in tidyverse data frames (`tibble`)



Data Manipulation Functions



Functions for Data Manipulation

Functions in the `dplyr` package:

- `%>%`: pipe operator, used to chain multiple operations together
- `glimpse()`: a glimpse into the data and its structure
- `filter()`: select observations (rows) that satisfy given conditions
- `arrange()`: reorder rows based on variable values
- `select()`: choose a subset of variables (columns)
- `mutate()`: create new variables or transform existing ones
- `summarise()`: reduce multiple values to a single summary
- `group_by()`: group data so operations are performed within each group



Filter: Introduction

```
1 #The first argument is the name of the data frame.
2 #The subsequent arguments are the expressions that filter the data frame.
3 filter(flights, month == 1, day == 1)
```

year <int>	month <int>	day <int>	dep_time <int>	sched_dep_time <int>	dep_delay <dbl>
2013	1	1	517	515	2
2013	1	1	533	529	4
2013	1	1	542	540	2
2013	1	1	544	545	-1
2013	1	1	554	600	-6
2013	1	1	554	558	-4
2013	1	1	555	600	-5
2013	1	1	557	600	-3
2013	1	1	557	600	-3
2013	1	1	558	600	-2

1-10 of 842 rows | 1-6 of 19 columns

Previous 1 2 3 4 5 6 ... 85 Next

```
1 # use the assignment operator, <- to save the result
2 #jan1 <- filter(flights, month == 1, day == 1)
3 # Save and print the result at the same time
4 #(dec25 <- filter(flights, month == 12, day == 25))
```

See also: Full Data



Filter: Comparisons

To use `filter()` effectively, you need to know how to select observations using comparison operators. R provides the standard set:

- `>` greater than
- `>=` greater than or equal to
- `<` less than
- `<=` less than or equal to
- `!=` not equal to
- `==` equal to

Note: be cautious when using `==` with floating point numbers. Consider using `near()` instead.



Filter: Logical Operators

- Multiple arguments in `filter()` are combined with “and”: every condition must be true for a row to be included in the output.

Other logical operators:

- `&`: “and”
- `|`: “or”
- `!`: “not”
- `x %in% y` select rows where `x` is one of the values in `y`

According to De Morgan’s laws:

- `!(x & y)` is equivalent to `!x | !y`
- `!(x | y)` is equivalent to `!x & !y`



Exercise: Filtering Multiple Months

Question:

Find all flights that departed in November or December.

```
1 filter(flights, month %in% c(11, 12))
```

year <int>	month <int>	day <int>	dep_time <int>	sched_dep_time <int>	dep_delay <dbl>
2013	11	1	5	2359	6
2013	11	1	35	2250	105
2013	11	1	455	500	-5
2013	11	1	539	545	-6
2013	11	1	542	545	-3
2013	11	1	549	600	-11
2013	11	1	550	600	-10
2013	11	1	554	600	-6
2013	11	1	554	600	-6
2013	11	1	554	600	-6

1-10 of 10,000 rows | 1-6 of 19 columns

Previous 1 2 3 4 5 6 ... 100 Next

```
1 #Alternatively, use the following
2 #filter(flights, month == 11 | month == 12)
```



Exercise: Combining Logical Conditions

Question:

Find flights that were not delayed (on arrival or departure) by more than two hours.

```
1 filter(flights, !(arr_delay > 120 | dep_delay > 120))
```

year <int>	month <int>	day <int>	dep_time <int>	sched_dep_time <int>	dep_delay <dbl>
2013	1	1	517	515	2
2013	1	1	533	529	4
2013	1	1	542	540	2
2013	1	1	544	545	-1
2013	1	1	554	600	-6
2013	1	1	554	558	-4
2013	1	1	555	600	-5
2013	1	1	557	600	-3
2013	1	1	557	600	-3
2013	1	1	558	600	-2

1-10 of 10,000 rows | 1-6 of 19 columns

Previous 1 2 3 4 5 6 ... 100 Next

```
1 #Alternatively, use the following
2 #filter(flights, arr_delay <= 120, dep_delay <= 120)
```



Filter: Missing Values

- **NA** represents an unknown value, so missing values are “contagious”: almost any operation involving an unknown value will also return **NA**.
- **filter()** only includes rows where the condition is **TRUE**; it excludes both **FALSE** and **NA** values. If you want to preserve missing values, you need to include them explicitly:

```
1 df <- tibble(x = c(1, NA, 3))
2 filter(df, x > 1)
```



	x
	<dbl>
	3
1 row	

```
1 filter(df, is.na(x) | x > 1)
```



	x
	<dbl>
	NA
	3
2 rows	



Arrange

- `arrange()`: order rows by column names (or more complex expressions)
- Use `desc()` to sort a column in descending order
- Missing values are always placed at the end

```
1 arrange(flights, year)
```

year <int>	month <int>	day <int>	dep_time <int>	sched_dep_time <int>	dep_delay <dbl>
2013	1	1	517	515	2
2013	1	1	533	529	4
2013	1	1	542	540	2
2013	1	1	544	545	-1
2013	1	1	554	600	-6
2013	1	1	554	558	-4
2013	1	1	555	600	-5
2013	1	1	557	600	-3
2013	1	1	557	600	-3
2013	1	1	558	600	-2

1-10 of 10,000 rows | 1-6 of 19 columns

Previous [1](#) [2](#) [3](#) [4](#) [5](#) [6](#) ... [1000](#) Next

```
1 #arrange(flights, desc(dep_delay))
```



Select

- `select()`: select a subset of variables (columns) by name

```
1 select(flights, year, month, day)
```

	year <int>	month <int>	day <int>
	2013	1	1
	2013	1	1
	2013	1	1
	2013	1	1
	2013	1	1
	2013	1	1
	2013	1	1
	2013	1	1
	2013	1	1
	2013	1	1

1-10 of 10,000 rows

Previous 1 2 3 4 5 6 ... 1000 Next

```
1 #select(flights, year:day)
2 #select(flights, -(year:day))
```



Select: Useful Functions

- `starts_with("abc")`: select variables whose names begin with “abc”
- `ends_with("xyz")`: select variables whose names end with “xyz”
- `contains("ijk")`: select variables whose names contain “ijk”.
- `matches()`: select variables that match a regular expression.
- `everything()`: select all variables

Other related functions:

- `rename()`: rename variables
- `mutate()`: create new variables from existing variables



Summarise

```
1 summarise(flights, delay = mean(dep_delay, na.rm = TRUE))
```



		delay
		<dbl>
		12.63907
1 row		



Summarise with Group-by

- This changes the unit of analysis from the entire dataset to individual groups.

```
1 by_day <- group_by(flights, year, month, day)
2 summarise(by_day, delay = mean(dep_delay, na.rm = TRUE))
```

year <int>	month <int>	day <int>	delay <dbl>
2013	1	1	11.54892601
2013	1	2	13.85882353
2013	1	3	10.98783186
2013	1	4	8.95159516
2013	1	5	5.73221757
2013	1	6	7.14801444
2013	1	7	5.41720430
2013	1	8	2.55307263
2013	1	9	2.27647715
2013	1	10	2.84499462

1-10 of 365 rows

Previous 1 2 3 4 5 6 ... 37 Next



Exercise: Summarising Flight Delays by Destination

- Explore the relationship between distance and average delay for each destination
- There are three steps to prepare this data:
 - Group flights by destination
 - Summarise to compute distance, average delay, and number of flights
 - Filter to remove noisy points (counts less than or equal to 20) and Honolulu (“HNL”) airport, which is much farther away than other destinations

Please have a try!



Solution: Data Manipulation

```

1 # Step 1: Group flights by destination
2 by_dest <- group_by(flights, dest)
3 # Step 2: Summarise to compute statistics for each destination
4 delay_summary <- summarise(by_dest,
5   count = n(),                                # Number of flights
6   dist = mean(distance, na.rm = TRUE),         # Average distance
7   delay = mean(arr_delay, na.rm = TRUE)        # Average arrival delay
8 )
9 # Step 3: Filter to remove noisy points and outliers
10 delay_summary <- filter(delay_summary,
11   count > 20,                                # Keep destinations with >20 flights
12   dest != "HNL")                             # Remove Honolulu (outlier: very far)
13
14 # Create visualization: relationship between distance and delay
15 # It looks like delays increase with distance up to ~750 miles
16 # and then decrease. Maybe as flights get longer there's more
17 # ability to make up delays in the air?
18 delay_plot <- ggplot(data = delay_summary,
19   mapping = aes(x = dist, y = delay)) +
20   geom_point(aes(size = count), alpha = 1/3) + # Point size represents flight count
21   geom_smooth(se = FALSE)                     # Add smooth trend line
22 #> `geom_smooth()` using method = 'loess' and formula 'y ~ x'

```



Solution: Visualisation

```
1 print(delay_plot)
```



Alternative Solution Using the Pipe Operator %>%

```
1 # Same analysis using pipe operator for cleaner, more readable code
2 delays_summary <- flights %>%
3   group_by(dest) %>%                                # Group by destination
4   summarise(
5     count = n(),                                     # Number of flights per destination
6     dist = mean(distance, na.rm = TRUE),             # Average distance
7     delay = mean(arr_delay, na.rm = TRUE)           # Average arrival delay
8   ) %>%
9   filter(count > 20, dest != "HNL")                 # Filter: >20 flights, exclude Honolulu
```



Missing values

- `na.rm=TRUE`: removes missing values

```
1 flights %>%
2   group_by(year, month, day) %>%
3   summarise(mean = mean(dep_delay, na.rm = TRUE))
```

year <int>	month <int>	day <int>	mean <dbl>
2013	1	1	11.54892601
2013	1	2	13.85882353
2013	1	3	10.98783186
2013	1	4	8.95159516
2013	1	5	5.73221757
2013	1	6	7.14801444
2013	1	7	5.41720430
2013	1	8	2.55307263
2013	1	9	2.27647715
2013	1	10	2.84499462

1-10 of 365 rows

Previous 1 2 3 4 5 6 ... 37 Next



Useful Summary Functions

- Measures of location (central tendency): `mean(x)`, `median(x)`
- Measures of spread (variability): `sd(x)`, `IQR(x)`
- Measures of rank: `min(x)`, `quantile(x, 0.25)`, `max(x)`
- Measures of position: `first(x)`, `nth(x, 2)`, `last(x)`
- Counts: `n()`, `sum(!is.na(x))`, `n_distinct(x)`
 - `count(tailnum, wt = distance)`: “count” (sum) the total number of miles a plane flew
- Counts and proportions of logical values: `sum(x > 10)`, `mean(y == 0)`

Note:

The interquartile range (IQR) is a measure of variability based on dividing a data set into quartiles. Q1 is the middle value in the lower half of the rank-ordered data. Q2 is the median value in the dataset. Q3 is the middle value in the upper half of the rank-ordered data. The interquartile range is equal to $Q3 - Q1$.



Grouping by Multiple Variables

```

1 # Group flights by date (year, month, day)
2 daily <- group_by(flights, year, month, day)
3 # Summarise: count flights per day
4 # n() returns the size of the current group
5 (per_day <- summarise(daily, flights = n()))

```

year <int>	month <int>	day <int>	flights <int>
2013	1	1	842
2013	1	2	943
2013	1	3	914
2013	1	4	915
2013	1	5	720
2013	1	6	832
2013	1	7	933
2013	1	8	899
2013	1	9	902
2013	1	10	932

1-10 of 365 rows

Previous 1 2 3 4 5 6 ... 37 Next



Ungrouping

```
1 # Remove grouping to get overall summary
2 daily %>%
3   ungroup() %>%           # Remove grouping structure
4   summarise(flights = n()) # Now counts all flights (not per day)
```

**flights**

<int>

336776

1 row



Relational Data



Relational Data

- **Relational data:** data stored across multiple tables that are related to each other

Three families of verbs designed to work with relational data:

- **Mutating joins:** add new variables to one data frame from matching observations in another
- **Filtering joins:** filter observations from one data frame based on whether they match observations in another table
- **Set operations:** treat observations as if they were set elements.
- A similar database system: SQL



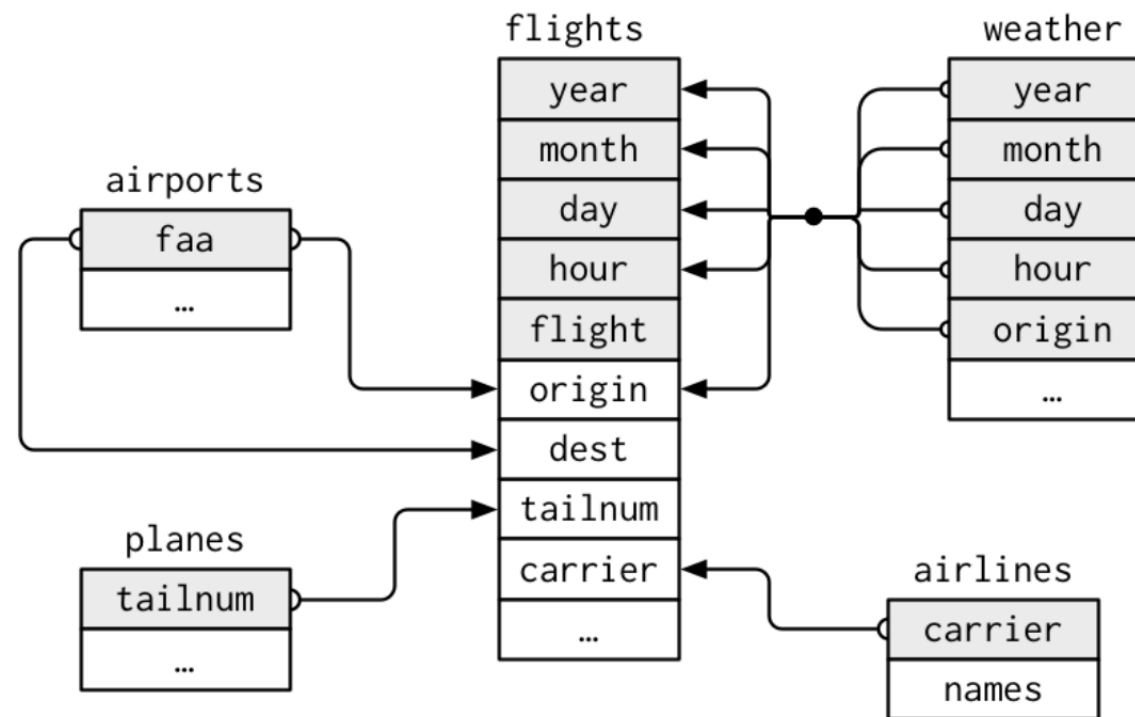
Dataset

- `nycflights13` contains five related tibbles:
 - `flights`: information about each flight
 - `airlines`: full carrier names
 - `airports`: information about each airport, identified by the `faa` airport code
 - `planes`: information about each plane, identified by its `tailnum`
 - `weather`: weather data at each NYC airport for each hour



Table Relationships

- Each relationship involves a pair of tables
- It is important to understand how tables are connected when working with relational data



Relations of Tibbles (Source: Wickham and Grolemund (2017), *R for Data Science*)



Keys

- **Primary key:** a variable (or set of variables) that uniquely identifies each observation in its own table.
 - For example, `planes$tailnum` is a primary key because it uniquely identifies each plane in the `planes` table.
- **Foreign key:** a variable (or set of variables) in one table that refers to a primary key in another table.
 - For example, `flights$tailnum` is a foreign key because it links each flight to a plane in the `planes` table.
- A variable can be both a primary key and a foreign key. - For example, `origin` is part of the primary key in the `weather` table, and is also a foreign key that links to the `airports` table.



Identify the primary keys

- Use `count()` the primary keys and look for entries where `n` is greater than 1.

```
1 # Check if tailnum is a valid primary key (should have no duplicates)
2 # If this returns empty, tailnum is a valid primary key
3 planes %>%
4   count(tailnum) %>%           # Count occurrences of each tailnum
5   filter(n > 1)               # Find any duplicates (n > 1 means not unique)
```

0 rows



Add a Primary Key

- What is the primary key in the `flights` table?
 - None
- **Surrogate key**: add a primary key using `mutate()` and `row_number()`
- A primary key and the corresponding foreign key in another table form a **relationship**.
- Relationships are typically one-to-many. Many-to-many relationships can be represented using a many-to-one relationship plus a one-to-many relationship.



Exercise: Add a Surrogate Key to `flights`

```
1 flights %>%
2   arrange(year, month, day, sched_dep_time, carrier, flight) %>%
3   mutate(flight_id = row_number()) %>%
4   #This makes it possible to see every column in a data frame.
5   glimpse()
```

Rows: 336,776

Columns: 20

```
$ year      <int> 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2~
$ month     <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
$ day       <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
$ dep_time  <int> 517, 533, 542, 544, 554, 559, 558, 559, 558, 558, 557, ~
$ sched_dep_time <int> 515, 529, 540, 545, 558, 559, 600, 600, 600, 600, 600, ~
$ dep_delay <dbl> 2, 4, 2, -1, -4, 0, -2, -1, -2, -2, -3, NA, 1, 0, -5, --
$ arr_time  <int> 830, 850, 923, 1004, 740, 702, 753, 941, 849, 853, 838,~
$ sched_arr_time <int> 819, 830, 850, 1022, 728, 706, 745, 910, 851, 856, 846,~
$ arr_delay <dbl> 11, 20, 33, -18, 12, -4, 8, 31, -2, -3, -8, NA, -6, -7,~
$ carrier   <chr> "UA", "UA", "AA", "B6", "UA", "B6", "AA", "AA", "B6", "~
$ flight    <int> 1545, 1714, 1141, 725, 1696, 1806, 301, 707, 49, 71, 79~
$ tailnum   <chr> "N14228", "N24211", "N619AA", "N804JB", "N39463", "N708~
$ origin    <chr> "EWR", "LGA", "JFK", "JFK", "EWR", "JFK", "LGA", "LGA",~
$ dest      <chr> "IAH", "IAH", "MIA", "BQN", "ORD", "BOS", "ORD", "DFW",~
$ air_time  <dbl> 227, 227, 160, 183, 150, 44, 138, 257, 149, 158, 140, N~
$ distance  <dbl> 1400, 1416, 1089, 1576, 719, 187, 733, 1389, 1028, 1005~
$ hour      <dbl> 5, 5, 5, 5, 5, 5, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6~
$ minute    <dbl> 15, 29, 40, 45, 58, 59, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0~
$ time_hour <dtm> 2013-01-01 05:00:00, 2013-01-01 05:00:00, 2013-01-01 0~
$ flight_id <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, ~
```



Mutating Joins

- Mutating join combine variables from two tables. They match observations by their keys, then add variable from one table to the other.
- Add columns from **y** to **x**:
 - `inner_join()`: keep observations that appear in both tables
 - `left_join()`: keep all observations in **x**
 - `right_join()`: keep all observations in **y**
 - `full_join()`: keep all observations in both **x** and **y**



```

1 # Join flights with airline names
2 # Select relevant columns from flights, then join with airlines table
3 flights %>%
4   select(year:day, hour, tailnum, carrier) %>%
5   left_join(airlines, by = "carrier") # Join by carrier code (key)

```

year <int>	month <int>	day <int>	hour <dbl>	tailnum <chr>	carrier <chr>
2013	1	1	5	N14228	UA
2013	1	1	5	N24211	UA
2013	1	1	5	N619AA	AA
2013	1	1	5	N804JB	B6
2013	1	1	6	N668DN	DL
2013	1	1	5	N39463	UA
2013	1	1	6	N516JB	B6
2013	1	1	6	N829AS	EV
2013	1	1	6	N593JB	B6
2013	1	1	6	N3ALAA	AA

1-10 of 10,000 rows | 1-6 of 7 columns

Previous 1 2 3 4 5 6 ... 1000 Next



Mutating Joins: Key Columns

- If keys are duplicated, the join returns all possible combinations of matching rows.
- Defining key columns
 - `by=NULL`: uses all variables that appear in both tables (a natural join)
 - `by = "x"`: uses a common variable named "x"
 - `by = c("a" = "b")`: match variable **a** in table **x** to variable **b** in table **y**



Mutating Joins: `base::merge()`

<code>dplyr</code>	<code>merge</code>
<code>inner_join(x, y)</code>	<code>merge(x, y)</code>
<code>left_join(x, y)</code>	<code>merge(x, y, all.x = TRUE)</code>
<code>right_join(x, y)</code>	<code>merge(x, y, all.y = TRUE)</code>
<code>full_join(x, y)</code>	<code>merge(x, y, all.x = TRUE, all.y = TRUE)</code>

- `dplyr` joins are generally faster and preserve the order of rows.



Filtering Joins

- Filtering joins match observations in the same way as mutating joins, but they affect rows (observations), not columns (variables).
- There are two types:
 - `semi_join(x, y)`: keep all observations in `x` that have a match in `y`
 - `anti_join(x, y)`: drop all observations in `x` that have a match in `y`



Exercise: Using semi_join()



Question:

- Find the ten most popular destinations
- Match them back to `flights`

```

1 # Step 1: Find top 10 most popular destinations
2 top_dest <- flights %>%
3   count(dest, sort = TRUE) %>%      # Count flights per destination, sort descending
4   head(10)                          # Keep only top 10
5
6 # Step 2: Filter flights to only those going to top destinations
7 flights %>%
8   semi_join(top_dest)                # Keep only flights matching top destinations

```

year <int>	month <int>	day <int>	dep_time <int>	sched_dep_time <int>	dep_delay <dbl>
2013	1	1	542	540	2
2013	1	1	554	600	-6
2013	1	1	554	558	-4
2013	1	1	555	600	-5
2013	1	1	557	600	-3
2013	1	1	558	600	-2
2013	1	1	558	600	-2
2013	1	1	558	600	-2
2013	1	1	559	559	0
2013	1	1	600	600	0

1-10 of 10,000 rows | 1-6 of 19 columns

Previous 1 2 3 4 5 6 ... 100 Next



Exercise: Using anti_join()

Question:

When connecting `flights` and `planes`, which flights do not have a match in `planes`?

```
1 # Find flights with tailnums that don't exist in planes table
2 # These are flights with missing plane information
3 flights %>%
4   anti_join(planes, by = "tailnum") %>% # Keep flights NOT in planes table
5   count(tailnum, sort = TRUE)          # Count occurrences of each unmatched tailnum
```

tailnum	n
<chr>	<int>
NA	2512
N725MQ	575
N722MQ	513
N723MQ	507
N713MQ	483
N735MQ	396
N0EGMQ	371
N534MQ	364
N542MQ	363
N531MQ	349

1-10 of 722 rows

Previous 1 2 3 4 5 6 ... 73 Next



Join Problems

- Start by identifying the variables that form the primary key in each table.
- Check that none of the variables in the primary key are missing. If a value is missing, it cannot identify an observation.
- Check that your foreign keys match primary keys in another table. The best way to do this is using `anti_join()`.



Set Operations

- `intersect(x, y)`: return observations that appear in both `x` and `y`
- `union(x, y)`: return all unique observations in `x` and `y`
- `setdiff(x, y)`: return observations in `x` but not in `y`



Examples of Set Operations

```

1 # Create two sample data frames
2 df1 <- tribble(
3   ~x, ~y,
4     1,  1,
5     2,  1
6 )
7 df2 <- tribble(
8   ~x, ~y,
9     1,  1,
10    1,  2
11 )
12 # Set operations
13 intersect(df1, df2)    # Rows in both df1 and df2

```

x	y
<dbl>	<dbl>
1	1

1 row

```

1 union(df1, df2)    # All unique rows from df1 and df2

```

x	y
<dbl>	<dbl>
1	1
2	1
1	2

3 rows



References

Wickham, Hadley, Mine Çetinkaya-Rundel, and Garrett Grolmund. 2023. *R for Data Science: Import, Tidy, Transform, Visualize, and Model Data*. 2nd ed. O'Reilly Media.

Wickham, Hadley, and Garrett Grolmund. 2017. *R for Data Science: Import, Tidy, Transform, Visualize, and Model Data*. Sebastopol, CA: O'Reilly Media.

